epoll(): I/O multiplexing ad alte prestazioni

Oltre i limiti di select()

Roberto Marino. PhD

Università di Messina
CdL Informatica — Reti e Sistemi Distribuiti Mod.B

Last Update: March 19, 2026

UniMe
1548

Perche' `select()` non basta

Il costo O(n) ad ogni chiamata

Il kernel scansiona **tutti** gli fd monitorati ad ogni ciclo, anche quelli idle.

Con 10.000 client e 3 attivi: 9.997 controlli inutili ad ogni iterazione.

Copia O(n) ad ogni chiamata

L'intero `fd_set` viene copiato da userspace a kernel ad ogni chiamata e restituito modificato.

Inoltre `fd_set` va ricostruito da zero ad ogni ciclo: tre operazioni O(n) per ogni iterazione del loop.

Il limite di `select()`

`FD_SETSIZE` $\approx$ 1024: non si possono monitorare piu' di $\sim$1024 fd contemporaneamente.

Op.	select	epoll
wait	O(n)	O(eventi)
add	O(n)	O(log n)
limite	1024	nessuno

L'idea chiave di `epoll()`

Il problema di `select()`

Non ricorda nulla tra una chiamata e l'altra. Ad ogni ciclo riceve l'intero insieme di fd, lo scansiona, lo sovrascrive e lo restituisce — O(n) sempre.

La soluzione di `epoll()`

Il kernel mantiene internamente un **albero rosso-nero** con tutti gli fd registrati.

`epoll_wait()` non scansiona nulla: restituisce *solo* gli fd che hanno avuto eventi, attingendo a una **ready list** aggiornata in tempo reale.

Costo: **O(eventi pronti)**, non O(fd totali).

L'idea chiave di `epoll()`

	select	**epoll**
Stato nel kernel	Nessuno	Albero rosso-nero + ready list
Registrazione fd	Ogni ciclo, O(n)	Una volta sola, O(log n)
Ritorno	Tutti gli fd (modificati)	Solo gli fd con eventi

Cos'e' un albero rosso-nero?

Definizione

Un albero rosso-nero e' un **albero binario di ricerca autobilanciante**. Ogni nodo contiene una chiave e viene colorato rosso o nero secondo regole precise che garantiscono il bilanciamento automatico dopo ogni inserimento e rimozione.

Albero binario di ricerca

In un albero binario di ricerca ogni nodo rispetta questa proprieta':

- Tutti i nodi nel sottoalbero **sinistro** hanno chiave **minore** del nodo corrente
- Tutti i nodi nel sottoalbero **destro** hanno chiave **maggiore** del nodo corrente

Questo permette di trovare qualsiasi chiave in O(log n) scendendo dall'albero — a patto che l'albero sia *bilanciato*.

Albero rosso-nero — le quattro regole

Le quattro regole che garantiscono il bilanciamento automatico:

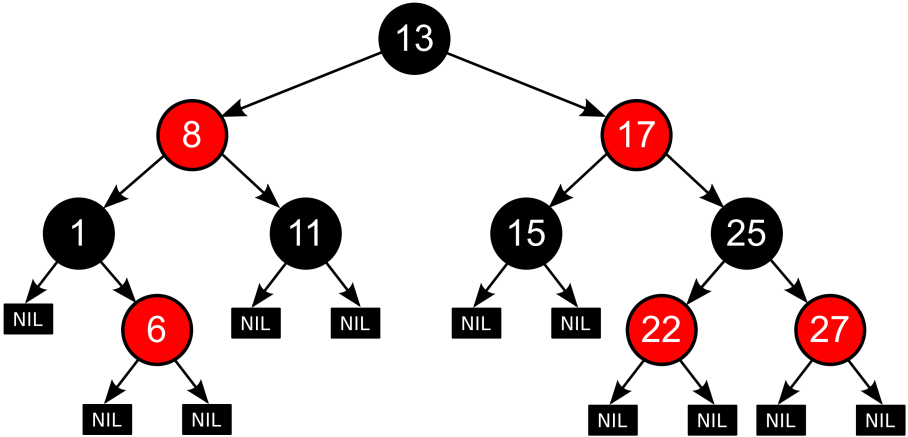
1. Ogni nodo e' colorato **rosso** o **nero**.
2. La **radice** e' sempre nera.
3. Nessun nodo **rosso** ha un figlio rosso (*no due rossi consecutivi*).
4. Ogni cammino dalla radice a una foglia attraversa lo stesso numero di **nodi neri** (*black-height uniforme*).

Conseguenza delle regole

Le quattro regole insieme garantiscono che l'altezza dell'albero sia sempre $h \leq 2\log_2(n+1)$, dove n e' il numero di nodi.

Questo significa che **ricerca, inserimento e rimozione** costano sempre **O(log n)** nel caso peggiore — indipendentemente dall'ordine in cui le chiavi vengono inserite.

Albero rosso-nero — le quattro regole



Albero rosso-nero — perche' lo usa `epoll()`

Il problema di `epoll()`

Il kernel deve gestire migliaia di fd registrati con `epoll_ctl()`.

Per ogni evento in arrivo deve trovare il fd corrispondente **velocemente**. Per ogni `ADD` e `DEL` deve inserire o rimuovere un nodo **velocemente**.

Un array richiederebbe O(n) per cerca e rimozione. Un albero rosso-nero garantisce O(log n) per tutte e tre le operazioni.

Op.	Array	RB-tree
Cerca fd	O(n)	O(log n)
ADD	O(1)	O(log n)
DEL	O(n)	O(log n)

Con 10.000 fd registrati

Array: fino a 10.000 confronti.
RB-tree: al massimo $2\log_2(10001) \approx$ **28 confronti**.

Le tre syscall di epoll()

```c
#include <sys/epoll.h>

/* 1. Crea l'istanza epoll nel kernel (ritorna un fd) */
int epfd = epoll_create1(0);

/* 2. Registra un fd --- UNA VOLTA SOLA */
struct epoll_event ev;
ev.events  = EPOLLIN;     /* evento di interesse  */
ev.data.fd = client_fd;   /* dato utente          */
epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);

/* 3. Attendi eventi: ritorna SOLO gli fd pronti */
struct epoll_event events[MAX_EVENTS];
int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
for (int i = 0; i < n; i++) {
    int fd = events[i].data.fd;
    recv(fd, buf, sizeof(buf), 0);
}
```

Le tre syscall di `epoll()` — panoramica

Tre syscall, tre ruoli, tre momenti diversi

Syscall	Ruolo	Quando	Costo
`epoll_create1()`	Crea la struttura nel kernel	Una volta sola	O(1)
`epoll_ctl()`	Registra / modifica / rimuove fd	A connect/disconnect	O(log n)
`epoll_wait()`	Aspetta e consegna gli eventi	Ad ogni ciclo	O(eventi)

La differenza fondamentale con `select()`

Con `select()` ricostruivi il set ad ogni ciclo — O(n) sempre.

Con `epoll()` la struttura dati persiste nel kernel tra una chiamata e l'altra: `epoll_wait()` non scansiona nulla, consegna direttamente cio' che e' nella ready list.

epoll_create1() — il pannello di controllo

```
/* Crea l'albero rosso-nero e la ready list nel kernel.
   Restituisce un fd che rappresenta l'istanza epoll. */
int epfd = epoll_create1(0);
if (epfd == -1) { perror("epoll_create1"); exit(1); }
```

Cosa crea nel kernel

- Un **albero rosso-nero** vuoto dove verranno registrati i fd
- Una **ready list** vuota dove il kernel inserisce i fd con eventi attivi
- Un **file descriptor** `epfd` per operare su questa struttura

Come si usa

- Si chiama **una volta sola**, prima del loop principale
- `epfd` si passa a tutte le chiamate successive
- Va chiuso con `close(epfd)` alla fine del programma
- Argomento `0`: nessun flag speciale

epoll_ctl(): registra, modifica, rimuovi

```c
struct epoll_event ev;
ev.events  = EPOLLIN;
ev.data.fd = fd;

/* Aggiunge fd al monitoraggio */
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);

/* Modifica gli eventi monitorati su fd gia' registrato */
ev.events = EPOLLIN | EPOLLOUT;
epoll_ctl(epfd, EPOLL_CTL_MOD, fd, &ev);

/* Rimuove fd dal monitoraggio */
epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
```

Flag events principali

EPOLLIN	dati in lettura
EPOLLOUT	pronto in scrittura
EPOLLERR	errore (sempre attivo)
EPOLLHUP	connessione chiusa
EPOLLET	edge-triggered

Il campo data

epoll_event.data e' una union: puoi usare data.fd per il fd oppure data.ptr per un puntatore a una struttura con lo stato della connessione.

epoll_ctl() — registra, modifica, rimuovi

```c
struct epoll_event ev;
ev.events  = EPOLLIN;     /* eventi da monitorare */
ev.data.fd = client_fd;   /* dato restituito all'evento */

/* Aggiunge fd all'albero */
epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);

/* Modifica gli eventi monitorati */
ev.events = EPOLLIN | EPOLLOUT;
epoll_ctl(epfd, EPOLL_CTL_MOD, client_fd, &ev);

/* Rimuove fd dall'albero */
epoll_ctl(epfd, EPOLL_CTL_DEL, client_fd, NULL);
```

Quando si chiama

ADD: quando un client si connette (dopo `accept()`).
MOD: se cambia il tipo di evento da monitorare.
DEL: quando un client si disconnette (prima di `close()`).

Mai dentro il loop critico — solo a connect e disconnect.

epoll_ctl() — il campo data

```c
/* epoll_event.data e' una union: scegli uno solo */
struct epoll_event {
    uint32_t     events;   /* eventi da monitorare */
    epoll_data_t data;     /* dato utente (union)  */
};

typedef union epoll_data {
    int      fd;     /* il file descriptor          */
    uint32_t u32;
    uint64_t u64;
    void    *ptr;    /* puntatore a struttura custom */
} epoll_data_t;
```

Perche' e' utile data.ptr

Associando un puntatore ad ogni fd, al momento dell'evento trovi subito lo stato della connessione senza cercare in strutture esterne.

epoll_wait() — aspetta e consegna

```
struct epoll_event events[MAX_EVENTS];

int n = epoll_wait(epfd,        /* istanza epoll          */
                   events,      /* array output           */
                   MAX_EVENTS,  /* max eventi per ciclo   */
                   -1);         /* timeout in ms (-1 = infinito) */
```

Il valore di ritorno n

- n > 0 — n fd pronti, scritti in events[0..n-1]
- n == 0 — timeout scaduto, nessun evento
- n < 0 — errore (errno == EINTR: segnale ricevuto, riprova)

Solo i pronti

A differenza di select(), events[] contiene **solo** gli fd con eventi attivi. Non devi controllare tutti gli fd con FD_ISSET: scorri da 0 a n-1 e sono tutti pronti.

epoll_wait() — come scorrere gli eventi

```c
int n = epoll_wait(epfd, events, MAX_EVENTS, -1);

for (int i = 0; i < n; i++) {
    int      fd  = events[i].data.fd;
    uint32_t evt = events[i].events;

    if (evt & EPOLLIN) {
        /* dati disponibili in lettura */
        recv(fd, buf, sizeof(buf), 0);
    }
    if (evt & EPOLLHUP) {
        /* connessione chiusa dall'altro lato */
        epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
        close(fd);
    }
    if (evt & EPOLLERR) {
        /* errore sul socket */
        epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
        close(fd);
    }
}
```

EPOLLERR e EPOLLHUP

Questi due eventi sono **sempre monitorati** da epoll, anche se non li specifichi in `ev.events`. Gestiscili sempre nel loop per evitare fd zombi.

Le tre syscall insieme — flusso completo

```c
/* 1. UNA VOLTA: crea la struttura nel kernel */
int epfd = epoll_create1(0);
struct epoll_event ev, events[MAX_EVENTS];

/* 2. UNA VOLTA: registra server_fd */
ev.events = EPOLLIN; ev.data.fd = server_fd;
epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

while (1) {
    /* 3. AD OGNI CICLO: consegna solo i pronti */
    int n = epoll_wait(epfd, events, MAX_EVENTS, -1);

    for (int i = 0; i < n; i++) {
        int fd = events[i].data.fd;
        if (fd == server_fd) {
            int cfd = accept(server_fd, NULL, NULL);
            ev.events = EPOLLIN; ev.data.fd = cfd;
            epoll_ctl(epfd, EPOLL_CTL_ADD, cfd, &ev);
        } else {
            ssize_t r = recv(fd, buf, sizeof(buf)-1, 0);
            if (r <= 0) {
                epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                close(fd);
            }
        }
    }
}
/* 4. ALLA FINE: chiudi l'istanza */
close(epfd);
```

Server multi-client con epoll() — struttura

```c
int epfd = epoll_create1(0);

/* Registra server_fd: una volta sola */
struct epoll_event ev;
ev.events = EPOLLIN; ev.data.fd = server_fd;
epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

while (1) {
    struct epoll_event events[MAX_EVENTS];
    int n = epoll_wait(epfd, events, MAX_EVENTS, -1);

    for (int i = 0; i < n; i++) {
        int fd = events[i].data.fd;

        if (fd == server_fd) {
            int cfd = accept(server_fd, NULL, NULL);
            ev.events = EPOLLIN; ev.data.fd = cfd;
            epoll_ctl(epfd, EPOLL_CTL_ADD, cfd, &ev);
        } else {
            char buf[1024];
            ssize_t r = recv(fd, buf, sizeof(buf)-1, 0);
            if (r <= 0) {
                epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
                close(fd);
            } else { buf[r]='\0'; printf("[%d] %s\n",fd,buf); }
        }
    }
}
```

Level-triggered vs Edge-triggered

Level-triggered (LT) — default

`epoll_wait()` notifica **finche'** il buffer ha dati non letti.

Se `recv()` legge solo parte dei dati, al ciclo successivo `epoll_wait()` notifica di nuovo.

Semplice da usare, non richiede socket non bloccanti. Compatibile con il modello `select()`.

Edge-triggered (ET) — `EPOLLET`

`epoll_wait()` notifica **una sola volta** alla transizione idle → pronto.

Devi leggere in loop fino a `EAGAIN`, altrimenti i dati rimangono nel buffer senza ulteriori notifiche.

Richiede socket non bloccanti. Piu' efficiente.

```
/* Level-triggered (default) */
ev.events = EPOLLIN;

/* Edge-triggered */
ev.events = EPOLLIN | EPOLLET;
/* il socket DEVE essere O_NONBLOCK */
```

Edge-triggered: loop di lettura corretto

```c
/* In ET devi leggere TUTTO fino a EAGAIN */
while (1) {
    ssize_t n = recv(fd, buf, sizeof(buf), 0);

    if (n > 0) {
        process(buf, n);            /* elabora */

    } else if (n == 0) {
        epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);
        close(fd);                  /* disconnesso */
        break;

    } else {                        /* n == -1 */
        if (errno == EAGAIN ||
            errno == EWOULDBLOCK) {
            break; /* buffer svuotato: aspetta il
                      prossimo evento epoll      */
        }
        perror("recv"); break;      /* errore vero */
    }
}
```

[!] Non uscire prima di EAGAIN

In ET, se esci dal loop senza aver letto tutto il buffer, `epoll_wait()` non ti notifichera' piu' per quei dati rimasti: li perdi silenziosamente.

Il campo data.ptr: stato per connessione

```c
/* Struttura con lo stato di ogni connessione */
typedef struct {
    int  fd;
    char buf[4096];     /* buffer di lettura parziale  */
    int  buf_len;       /* byte gia' ricevuti          */
    char addr[INET_ADDRSTRLEN];
} conn_t;

/* Registrazione: associa il puntatore all'evento */
conn_t *c = malloc(sizeof(conn_t));
c->fd = client_fd;
inet_ntop(AF_INET, &addr.sin_addr, c->addr, sizeof(c->addr));

struct epoll_event ev;
ev.events   = EPOLLIN;
ev.data.ptr = c;         /* puntatore, non fd! */
epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);

/* Recupero al momento dell'evento */
conn_t *c = (conn_t *)events[i].data.ptr;
recv(c->fd, c->buf + c->buf_len, ..., 0);
```

Perche' e' utile

Associando un puntatore ad ogni fd, il gestore dell'evento trova subito lo stato della connessione senza cercare in strutture dati esterne.

EPOLLONESHOT: un evento alla volta

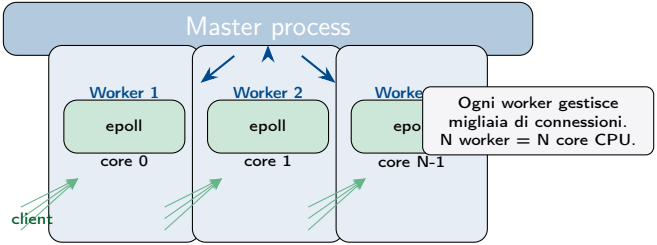
```
/* EPOLLONESHOT: dopo la prima notifica, il fd
   viene disabilitato automaticamente.
   Riabilitarlo con EPOLL_CTL_MOD.            */
ev.events = EPOLLIN | EPOLLET | EPOLLONESHOT;
epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev);

/* ... gestisci l'evento ... */

/* Riabilita per la prossima notifica */
ev.events   = EPOLLIN | EPOLLET | EPOLLONESHOT;
ev.data.ptr = c;
epoll_ctl(epfd, EPOLL_CTL_MOD, fd, &ev);
```

Quando usare EPOLLONESHOT

In un server multi-thread dove piu' thread chiamano `epoll_wait()` sullo stesso `epfd`, lo stesso evento potrebbe essere consegnato a due thread contemporaneamente.

`EPOLLONESHOT` garantisce che ogni evento venga gestito da un solo thread alla volta.

Architettura moderna: N worker + epoll



Nginx, Redis, Node.js

Nginx: 1 worker per core, ognuno con il proprio epoll loop.
Redis: 1 solo worker con epoll (I/O bound, no CPU parallela).
Node.js: libuv usa epoll su Linux sotto il cofano.

Complessita': il quadro completo

Operazione	select	epoll	io_uring
Attesa eventi	O(n)	O(eventi)	O(1)
Add fd	O(n)/ciclo	O(log n) una volta	O(1) una volta
Del fd	O(n)/ciclo	O(log n)	O(1)
Copia kernel	O(n)/ciclo	nessuna	nessuna
Limite fd	∼1024	nessuno	nessuno
Portabilita'	POSIX	Linux only	Linux only

Il log n di epoll_ctl

`epoll_ctl()` costa O(log n) perche' inserisce il fd in un **albero rosso-nero**. Ma si chiama raramente — solo alla connessione e disconnessione del client. `epoll_wait()` invece costa O(eventi pronti): e' la chiamata del loop critico.

Riepilogo: cinque cose da ricordare

1. **Tre syscall, ruoli distinti.** `epoll_create1()` una volta, `epoll_ctl()` per ogni connessione/disconnessione, `epoll_wait()` ad ogni ciclo.
2. **Registrazione unica.** A differenza di `select()`, il fd si registra *una volta sola*. Il kernel mantiene lo stato internamente nell'albero rosso-nero.
3. **Ritorna solo i pronti.** `epoll_wait()` consegna direttamente gli eventi attivi — O(eventi), non O(fd totali).
4. **LT e' piu' semplice, ET e' piu' efficiente.** Level-triggered si comporta come `select()`: facile da usare. Edge-triggered richiede socket non bloccanti e loop `EAGAIN`.
5. **Solo Linux.** `epoll` non e' POSIX. Su macOS/BSD si usa `kqueue`; su Windows `IOCP`. Per portabilita' si usano librerie come `libuv` o `libevent`.